Convie Multivendor Marketplace PRD

Oct 18, 2026 · @Prince Efik

Summary

Convie becomes a multivendor marketplace where independent vendors list products, buyers in several countries check out once across many vendors, and each vendor is paid out in their own currency. It is built on the existing Laravel 13 microservices (auth, user, product) behind one nginx gateway, adding vendor, cart, order, payment, payout and notification services.

The success bar for launch: a buyer in Lagos, Accra or London can pay for a two-vendor basket in their local currency, and both vendors receive the correct net amount in their local bank account within their settlement window, with every kobo traceable in the ledger.

Assumptions this PRD makes (confirm or correct):

- Launch markets are Nigeria, Ghana and Kenya for vendors and buyers, plus UK, EU and US buyers (diaspora demand).
- Gateways are Paystack (NG, GH, KE, ZA), Flutterwave (pan-African and multi-currency) and Stripe (UK, EU, US cards).
- Convie is the merchant of record and collects a commission per order; it does not hold buyer wallets in phase 1.

Goals, non-goals and success metrics

Goals

1. Vendors self-onboard, pass KYC/KYB and publish a catalogue without staff help.
2. Buyers check out a multi-vendor cart in one payment, in their own currency.
3. Each payment is split automatically into vendor net, Convie commission, gateway fee and tax.
4. Vendors are paid out on a predictable schedule to local bank or mobile money.
5. Every money movement is recorded in a double-entry ledger and reconciles daily against gateway reports.

Non-goals (phase 1)

- Buyer wallets, stored value or credit.
- Crypto or BNPL payment methods.

- Convie-run logistics; vendors ship themselves or via a third-party carrier API.
- Native mobile apps; the web front end is responsive.

Success metrics (first 6 months after GA)

Metric	Target	Why it matters
Payment success rate (authorised / attempted)	⩾ 92% NG, ⩾ 95% UK/EU	Gateway routing quality
Checkout conversion (cart → paid)	⩾ 3.5%	Core funnel health
Vendor onboarding time (sign-up → first live product)	⩽ 48 h median	Supply growth
Payout on time (within stated window)	⩾ 99%	Vendor trust
Daily reconciliation breaks	0 unresolved after 48 h	Financial integrity
Active vendors	300	Marketplace depth
GMV	$250k cumulative	Business viability
Dispute / chargeback rate	< 0.5% of orders	Gateway standing

Users

Persona	Where	Needs	Top pain today
Buyer	NG, GH, KE, UK, EU, US	Find products, pay in own currency with a familiar method (card, bank transfer, USSD, M-Pesa), track orders	Cross-border card declines; unclear landed price
Vendor	NG, GH, KE	List fast, get paid reliably in local currency, see fees clearly	Late or opaque payouts on other marketplaces
Operations admin	Convie	Approve vendors, moderate listings, handle disputes and refunds	No tooling yet

Persona	Where	Needs	Top pain today
Finance	Convie	Reconcile gateway settlements, run payouts, report tax and FX	No ledger yet

Where the codebase stands today

Three services run today behind nginx, each with its own MySQL 8.4 database. There is nothing yet for vendors, orders or payments. `develop` is at `618d17c` (5 weeks old). The product API lives on two unmerged branches.

Component	State	Notes
`auth-service`	Working	Register, login, logout, `/auth/user` on Sanctum tokens. `/internal/auth/validate` is guarded by an `X-Service-Key` shared secret. No roles, email verification or password reset.
`user-service`	Working	`/users/me` returns phone, avatar and bio. Calls auth-service on every request to validate the token.
`product-service`	Branches only	`feature/product-service-api` (b23ea35): public list and search, soft deletes, slug, `currency` defaulting to NGN, rate limits on auth. `feat/product-service-wiring` (c05860a): simpler CRUD with no currency. The two overlap and conflict, so one must be chosen.
`convie` (monolith shell)	Scaffold	Default Laravel app served on `/`. Its role is undefined: front end, BFF or retired.
Gateway	nginx	Path routing to php-fpm per service. No rate limiting, TLS or request IDs.
Async messaging	None	Redis runs but is unused. No events between services.
CI and tests	Minimal	Feature tests exist on the product branches only.

Gaps that block the marketplace: no roles (buyer, vendor, admin), no vendor entity, product prices with no multi-currency model, no events bus, and token validation as a synchronous call on every request.

Functional requirements

P0 is required for launch, P1 is required within 90 days of launch, and P2 is planned later.

ID	Area	Requirement	Owner service	Priority
FR-01	Identity	Roles: buyer, vendor_owner, vendor_staff, admin, finance. Roles are carried in token claims.	auth	P0
FR-02	Identity	Email and phone verification, password reset, optional TOTP 2FA (required for vendors and admins)	auth	P0
FR-03	Vendor	Vendor application with business details, country, settlement currency and bank or mobile-money account	vendor (new)	P0
FR-04	Vendor	KYC/KYB using BVN or NIN (NG), Ghana Card (GH) or KRA PIN (KE), with an approve or reject workflow	vendor	P0
FR-05	Vendor	Gateway subaccount created per vendor (Paystack or Flutterwave) or connected account (Stripe Connect)	payment (new)	P0
FR-06	Catalogue	Products belong to a vendor and have variants (size, colour), SKU, stock, images and a category	product	P0
FR-07	Catalogue	Base price in the vendor's currency, with display price converted at a quoted FX rate	product + pricing	P0
FR-08	Catalogue	Search, filter (price, category, vendor, ships-to country) and sort	product	P0
FR-09	Catalogue	Listings need admin approval for the first 10 products of a new vendor	product	P1
FR-10	Cart	Persistent cart across sessions, grouped by vendor, with stock checked at add and at checkout	cart (new)	P0

ID	Area	Requirement	Owner service	Priority
FR-11	Checkout	One payment for a multi-vendor cart, with shipping fee per vendor and tax per destination	order (new)	P0
FR-12	Checkout	Price and FX rate locked for 15 minutes once checkout starts	pricing	P0
FR-13	Orders	One parent order splits into vendor sub-orders, each with its own status lifecycle	order	P0
FR-14	Orders	Vendor marks shipped with tracking. Buyer confirms receipt, or receipt auto-confirms after N days.	order	P0
FR-15	Payments	Gateway chosen by buyer country, currency and method (see next section)	payment	P0
FR-16	Payments	Webhooks verified by signature and processed idempotently. Status is always re-queried from the gateway before an order is marked paid.	payment	P0
FR-17	Payments	Full and partial refunds per vendor sub-order	payment	P0
FR-18	Payouts	Vendor balance moves from pending to available after the hold period, then is paid out on a schedule (weekly by default)	payout (new)	P0
FR-19	Ledger	Double-entry ledger records every capture, fee, commission, refund, payout and FX conversion	ledger (new)	P0
FR-20	Ledger	Daily reconciliation against gateway settlement reports, with breaks flagged	ledger	P0
FR-21	Disputes	Buyer opens a dispute, the vendor responds, an admin decides, and refund or release follows	order + admin	P1
FR-22	Notifications	Email and SMS for order, payment, shipping, payout and KYC events	notification (new)	P0

ID	Area	Requirement	Owner service	Priority
FR-23	Vendor dashboard	Sales, balance (pending and available), payouts, fees and a downloadable statement	vendor	P0
FR-24	Admin	Vendor approval, listing moderation, order lookup, refund override, payout hold	admin (new)	P0
FR-25	Reviews	Ratings and reviews limited to verified purchases	review (new)	P2

Cross-border payments

A new `payment-service` sits behind one internal interface (`PaymentGateway` : initialize, verify, refund, createSubaccount, payout). Each provider is an adapter behind that interface, and a routing table picks one per transaction. No other service calls a gateway directly.

Routing rules (phase 1)

Buyer country	Currency charged	Primary gateway	Fallback	Methods
Nigeria	NGN	Paystack	Flutterwave	Card, bank transfer, USSD
Ghana	GHS	Paystack	Flutterwave	Card, mobile money
Kenya	KES	Flutterwave	Paystack	M-Pesa, card
UK	GBP	Stripe*	Flutterwave	Card, Apple Pay, Google Pay
EU	EUR	Stripe*	Flutterwave	Card, Apple Pay, Google Pay
US	USD	Stripe*	Flutterwave	Card, Apple Pay, Google Pay

*Stripe requires Convie to hold a business entity in a Stripe-supported country, because Nigerian businesses cannot open Stripe accounts directly. Until then, international cards route through Flutterwave. This is open question Q1.

Money model

- Amounts are stored as integers in minor units (kobo, pesewa, cents) with an ISO 4217 currency code. Floats and `decimal(12,2)` are not used for money in the ledger.
- Each product has a base currency, which is the vendor's settlement currency.
- The buyer sees a converted price. The FX rate comes from a rate provider plus a configurable spread of 1.5% by default, and is locked for 15 minutes at checkout.
- The order stores four values per line: base amount, charged amount, rate ID and spread.

Split settlement

- If the buyer's currency equals the vendor's currency and the gateway supports splits (Paystack or Flutterwave subaccounts), the split happens at the gateway: vendor share goes to the subaccount and commission stays with Convie.
- If the currencies differ, Convie collects the full amount. The ledger records the vendor's share in the vendor's currency at the locked rate, and the payout-service pays out later through the gateway's transfer API. FX risk between lock and payout sits with Convie and is reported daily.
- Commission is a percentage per category (default 8%) plus an optional fixed fee. Gateway fees are borne by Convie in phase 1.

Payouts

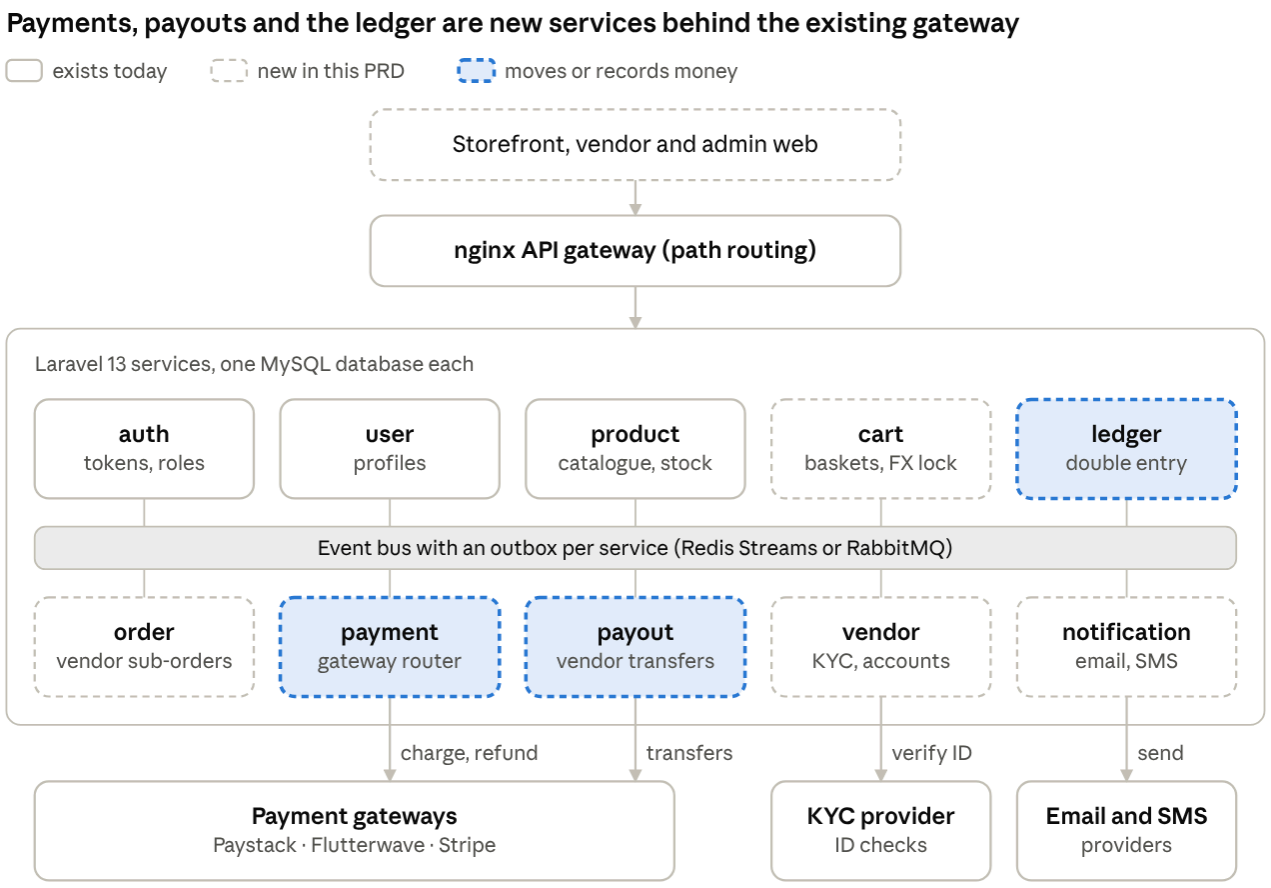
- Pending funds move to available after the delivery confirmation plus a hold period (7 days default, 14 days for new vendors).
- Payouts run weekly to a bank account (NG, GH, KE) or mobile money (GH, KE), with a minimum payout threshold per currency.
- Every payout has a reference, a ledger entry and a status callback. A failed payout reverts to the available balance and triggers an alert.

Non-functional requirements

ID	Category	Requirement
NFR-01	Security	PCI DSS SAQ-A scope only. Card data never touches Convie servers; hosted or inline gateway checkout only.

ID	Category	Requirement
NFR-02	Security	Service-to-service auth via signed JWT (auth-service issues it and services verify it locally with a public key), replacing the per-request `/internal/auth/validate` call. The shared `X-Service-Key` is kept only for internal admin calls and rotated quarterly.
NFR-03	Security	Secrets live in a secret manager, never in `.env` in git. Gateway keys are per environment, and live keys exist only in Production.
NFR-04	Data protection	NDPA 2023 (NG), Ghana DPA 2012, Kenya DPA 2019 and UK/EU GDPR. KYC documents are encrypted at rest, kept for the statutory period, then purged.
NFR-05	Reliability	Payment, order and ledger services reach 99.9% monthly availability. Webhook processing is at-least-once and idempotent by gateway event ID.
NFR-06	Consistency	Cross-service workflows (checkout, refund, payout) run as sagas over an event bus with an outbox table per service. No distributed transactions.
NFR-07	Performance	p95 latency: catalogue read 300 ms, add to cart 400 ms, checkout initialize 1.5 s excluding the gateway redirect
NFR-08	Observability	Request ID propagated through nginx and every service. Structured JSON logs and metrics per service. Alerts on payment success rate, webhook lag and reconciliation breaks.
NFR-09	Auditability	Ledger entries are append-only. Corrections are reversing entries. Admin actions on money are logged with actor and reason.
NFR-10	Localisation	Currency and number formatting per locale. English at launch, with copy externalised for French (francophone West Africa) later.

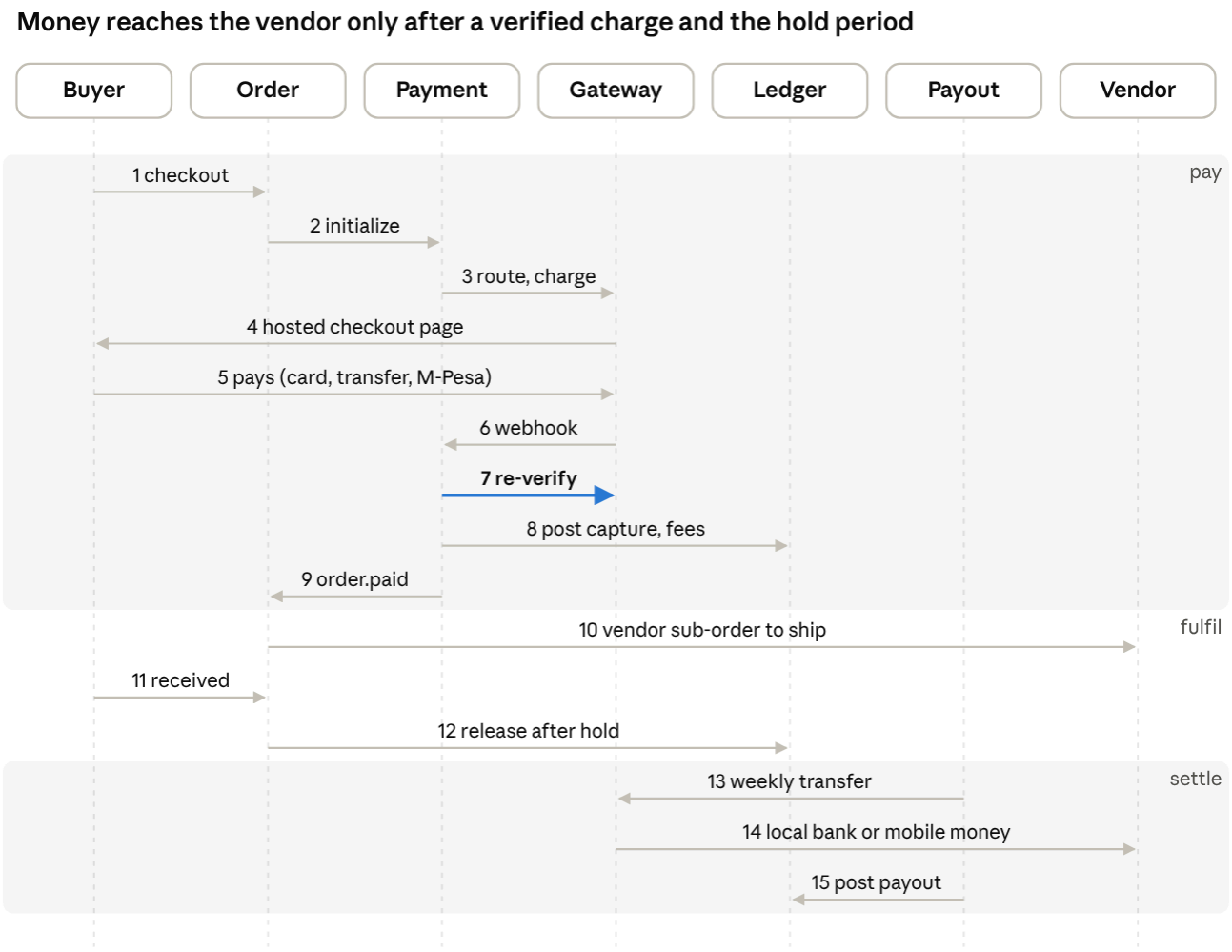
Target architecture



target architecture · 10 services, 3 external providers

Services only talk synchronously through the gateway for reads. Every state change that another service cares about (order paid, refund issued, payout settled) is published as an event, and gateway webhooks land only on the payment service.

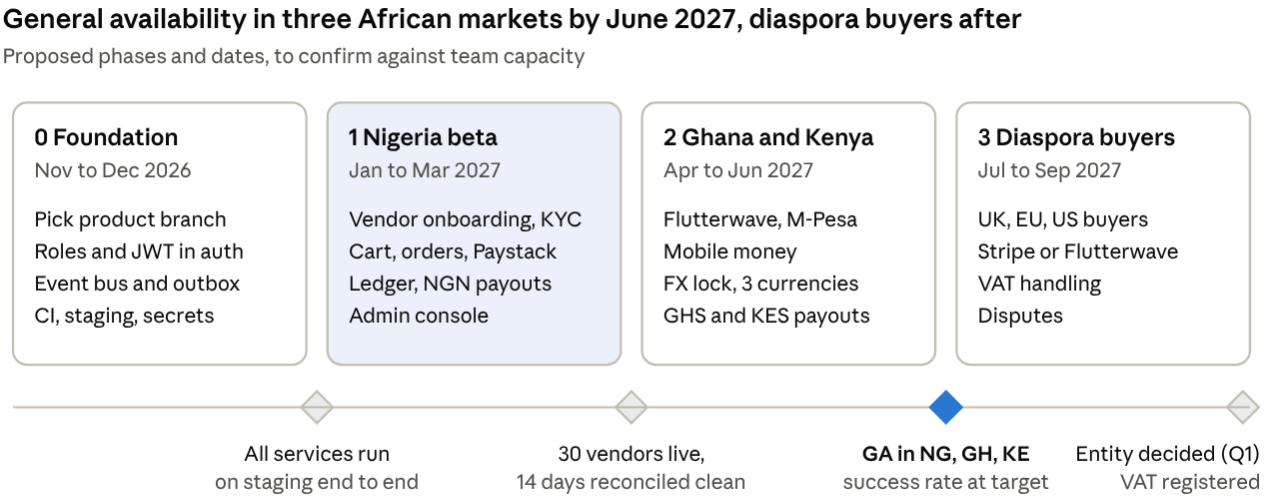
Checkout and payment flow



checkout to payout · 15 messages in 3 phases

Step 7 is the control that matters: a webhook alone never marks an order paid. Payment-service re-queries the gateway, checks amount and currency against the order, then posts to the ledger before any vendor is told to ship.

Release plan



release plan · 4 phases, 4 gates

Each phase ships to `staging` first and is promoted to `Production` only when its gate is met. Phase 1 keeps a single currency (NGN) so the ledger and payouts are proven before FX enters.

Risks and open questions

Risk	Likelihood	Impact	Mitigation
FX loss between rate lock and vendor payout	High	Medium	Spread on the quoted rate, short payout cycles, daily FX exposure report, and a cap on cross-currency volume per day
Gateway outage or decline spike in one market	Medium	High	Fallback gateway per route, with an automatic switch when success rate drops below the threshold over 15 minutes
Webhook loss or duplication leading to wrong order state	Medium	High	Idempotency keys, verify-before-fulfil, and a 5-minute reconciliation poller for pending payments
Vendor fraud (fake listings, non-delivery)	Medium	High	KYB, payout hold for new vendors, listing approval, buyer protection window

Risk	Likelihood	Impact	Mitigation
Regulatory change on FX repatriation (CBN) or payment licensing	Medium	High	Convie stays a merchant on licensed gateways rather than a payment service provider, with legal review before each new market
Microservice sprawl slowing a small team	High	Medium	Add services only where data ownership differs. Admin, notification and review can start inside existing services.

Open questions

- [ ] Q1: Will Convie register a UK or US entity to use Stripe, or route all international cards through Flutterwave?
- [ ] Q2: Which product branch is the base: `feature/product-service-api` or `feat/product-service-wiring`?
- [ ] Q3: What role does the `convie` app play: front end, BFF or retired?
- [ ] Q4: Who carries gateway fees and FX spread, Convie or the vendor? This affects pricing transparency.
- [ ] Q5: Is VAT collected per destination (UK, EU OSS) at launch, or are those buyers held back to phase 2?
- [ ] Q6: Is there an event bus choice? Redis Streams (already running) or RabbitMQ.